

Reviewer Pack — Minimal Rank of Noncommutative Geometry States vs DPLL Proof...

Entry 15ab436dc790 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 19:29:07 UTC

Minimal Rank of Noncommutative Geometry States vs DPLL Proof Length

Entry ID: 15ab436dc790 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 19:29:07 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Complexity Theory: DPLL Proof Length

Statement:

[‘For any given CNF formula F with n variables, the minimal rank of a noncommutative geometric state representing the logical structure of F is linearly proportional to the length of the shortest DPLL proof for F .’, ‘Equivalently, there exists a constant $c > 0$ such that for all CNF formulas F with n variables, $|R(F)| = O(c * t(F))$ where $R(F)$ denotes the minimal rank of the noncommutative geometric state and $t(F)$ is the length of the shortest DPLL proof.’]

Rationale (proposer’s reasoning):

[“Noncommutative geometry offers a unique framework for studying the algebraic structure of quantum systems, which could potentially provide insights into the complexity of logical operations like satisfiability. The conjecture suggests that the geometric representation of a CNF’s logical structure could be used to predict its proof length.”, ‘If true, this would bridge the gap between noncommutative geometry and computational complexity theory, providing a new perspective on the problem of satisfiability.’]

Taxonomy category: Noncommutative_Geometry_vs_Complexity_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a1f3c3a587c87b8a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean correlation coefficient between the minimal rank of noncommutative geometric states and the DPLL proof length is ≥ 0.8 , AND no seed produces a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative geometry' AND 'DPLL proof length' AND 'minimal rank' - 'geometric state' AND CNF formula AND 'shortest DPLL proof' - 'noncommutative geometry' related to 'proof complexity' via 'DPLL'

Top relevant hits considered: - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/1604.00499v1] From Monge to Higgs: a survey of distance computations in noncommutative geometry - [http://arxiv.org/abs/math/0205005v2] The rings of noncommutative projective geometry - [http://arxiv.org/abs/1506.01602v1] A Concurrency Problem with Exponential DPLL(T) Proofs - [http://arxiv.org/abs/2201.08895v4] On the Satisfaction Probabilities of k -CNF Formulas - [http://arxiv.org/abs/2403.13102v1] Geometric methods in quantum information and entanglement variational principle - [http://arxiv.org/abs/1209.3595v2] Noncommutative complex differential geometry - [http://arxiv.org/abs/1304.4062v2] Superconformal nets and noncommutative geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=242.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                continue
            clauses.append(clause)
        return clauses

    def dpll_length(cnf):
        n = len(cnf[0])
        stack = []
        assignment = [None] * (n + 1)
```

```

def backtrack():
    while True:
        if not stack:
            return len(assignment) - 1
        i, j = stack.pop()
        if assignment[j] is None:
            assignment[j] = 1
            for clause in cnf:
                if all(x == 0 or (x > 0 and assignment[x] == 1) or (x < 0 and assignment[-x] == -1) for x in clause):
                    break
            else:
                stack.append((i, j + 1))
                continue
        assignment[j] = -assignment[j]
        if all(x == 0 or (x > 0 and assignment[x] == 1) or (x < 0 and assignment[-x] == -1) for x in clause):
            stack.append((i, j + 1))
        else:
            stack.append((i, j + 1))
    return len(assignment) - 1

return backtrack()

def minimal_rank(cnf):
    n = len(cnf[0])
    rank = 0
    for i in range(n):
        if all(x == 0 or (x > 0 and assignment[x] == 1) or (x < 0 and assignment[-x] == -1) for x in cnf[i]):
            rank += 1
    return rank

n = random.randint(5, 40)
cnf = generate_cnf(n)
dpll_len = dpll_length(cnf)
min_rank = minimal_rank(cnf)

if dpll_len == 0:
    return {
        "metric_name": "minimal_rank_vs_dpll_length",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "DPLL proof length is zero"
    }

correlation = min_rank / dpll_len
return {
    "metric_name": "minimal_rank_vs_dpll_length",
    "metric_value": correlation,
    "instances_tested": 1,
    "conjecture_holds": correlation >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result["metric_value"])

mean = sum(results) / len(results)
std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
support_fraction = sum(1 for r in results if r >= 0.5) / len(results)

if all(r >= 0.8 for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r < 0.5 for r in results):
    first_failing_seed = seeds[results.index(min([r for r in results if r < 0.5]))]
    print(f"RESULT: FALSIFIED counterexample='correlation_below_0.5' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the analysis required to evaluate the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11467
2	propose	ollama_remote	glm4:latest	0	0	12362
3	preregistration	ollama_remote	glm4:latest	0	0	5495
4	novelty	ollama_remote	glm4:latest	0	0	4818
5	novelty	ollama_remote	glm4:latest	0	0	10023
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40377
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9816

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12390
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10785
10	judge	ollama_remote	glm4:latest	0	0	57763

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 175295 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/15ab436dc790.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/15ab436dc790.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/15ab436dc790.tar.gz (if generated)